Matplotlib Cheat Sheet

Sessions 1–11 | Every Function | Definition · When to Use · Code Example

S1 · Introduction — Figure Anatomy & Workflow

import matplotlib.pyplot as plt

Why: Standard import — gives you 'plt' shorthand to access all Matplotlib functions

```
import matplotlib.pyplot as plt
import numpy as np  # often used together
```

plt.figure(figsize=(w,h))

Why: Create a blank canvas. Use when you need custom size before adding any plots.

```
fig = plt.figure(figsize=(10, 5))
# (width, height) in inches
```

fig, ax = plt.subplots()

Why: Modern way — creates Figure + Axes together. PREFERRED for all plots.

```
fig, ax = plt.subplots(figsize=(8, 5))
ax.plot(x, y)  # use ax.method()
```

plt.show()

Why: Displays the plot in Jupyter/IDE. Always call at the end.

```
plt.plot(x, y)
plt.show()  # renders the plot
```

plt.clf() / plt.close()

Why: clf() clears current figure. close() closes window. Use to avoid overlapping plots.

```
plt.clf()   # clear current figure
plt.close() # close the window
```

Figure Anatomy: Figure (outer box) → Axes (plot area) → Axis (x/y lines) → Title, Labels, Ticks, Legend

S2 · Line Charts — plt.plot()

plt.plot(x, y, ...)

Why: Draw a line chart. Use for trends over time — sales, temperature, stock prices.

```
plt.plot(x, y)                          # basic
plt.plot(x, y, color='blue', linewidth=2)  # styled
plt.plot(x, y, marker='o', linestyle='--', label='Sales')
```

Markers

Why: Mark data points on the line. Use when data points matter, not just the trend.

```
marker='o'   # circle    marker='s'  # square
marker='^'   # triangle  marker='*'  # star
marker='D'   # diamond   marker='+'  # plus
```

Line Styles

Why: Control the line pattern. Dashed for forecast, dotted for target, solid for actual.

```
linestyle='-'    # solid (default)
linestyle='--'   # dashed
linestyle=':'    # dotted
linestyle='-.'   # dash-dot
```

Colors

Why: Set line/bar/scatter color. Use named colors, hex codes, or RGB tuples.

```
color='red'          # named
color='#FF5733'      # hex
color=(0.1,0.5,0.9)  # RGB tuple
```

Title & Labels

Why: Always add titles and axis labels so charts are self-explanatory.

```
plt.title('Monthly Sales', fontsize=14)
plt.xlabel('Month')  plt.ylabel('Revenue')
plt.xticks(rotation=45)  # rotate labels
```

S3 · Bar Charts — plt.bar() / plt.barh()

plt.bar(x, height, ...)

Why: Vertical bar chart. Use to compare categories — revenue by region, scores by team.

```
plt.bar(categories, values)              # basic
plt.bar(x, y, color='steelblue', width=0.6, edgecolor='black')
plt.bar(x, y, color=['red','blue','green'])  # different colors
```

plt.barh(y, width, ...)

Why: Horizontal bar chart. Better when category names are long or many bars exist.

```
plt.barh(categories, values)             # basic horizontal
plt.barh(y, x, color='coral', height=0.5)
```

Adding Text Labels on Bars

Why: Show values ON bars so viewers don't need to read the axis.

```
for i, val in enumerate(values):
    plt.text(i, val+0.5, str(val),
             ha='center', fontweight='bold')
```

Bar Width Control

Why: width param (0 to 1). Narrower bars for many categories, wider for few.

```
plt.bar(x, y, width=0.8)  # wider
plt.bar(x, y, width=0.3)  # narrower
# default width = 0.8
```

S4 · Pie & Donut Charts — plt.pie()

plt.pie(sizes, labels, ...)

Why: Show part-to-whole relationships. Use for market share, budget split, survey results.

```
plt.pie(sizes, labels=labels, autopct='%1.1f%%')
plt.pie(sizes, labels=labels, autopct='%1.1f%%',
        colors=['#FF6B6B','#4ECDC4','#45B7D1'],
        startangle=90, shadow=True)
```

autopct='%1.1f%%'

Why: Shows percentage inside each slice automatically. 1f = 1 decimal place.

```
autopct='%1.0f%%'  # 0 decimals: 25%
autopct='%1.1f%%'  # 1 decimal:  25.3%
autopct='%1.2f%%'  # 2 decimals: 25.34%
```

explode — Pull Out a Slice

Why: Highlight one slice by pulling it out. Good for emphasizing the biggest/smallest segment.

```
explode = (0.1, 0, 0, 0)  # pull 1st slice
plt.pie(sizes, explode=explode,
        labels=labels, autopct='%1.1f%%')
```

Donut Chart Technique

Why: A pie chart with a hole in the center. More modern look, often used in dashboards.

```
fig, ax = plt.subplots()
ax.pie(sizes, labels=labels, autopct='%1.1f%%')
centre_circle = plt.Circle((0,0), 0.70, fc='white')
ax.add_artist(centre_circle)  # creates the hole
```

S5 · Histogram — plt.hist()

plt.hist(data, bins, ...)

Why: Show distribution of continuous data — ages, prices, scores. Reveals shape of data.

```
plt.hist(data, bins=20)                          # basic
plt.hist(data, bins=30, color='steelblue',
         edgecolor='black', alpha=0.7)
plt.hist(data, bins=20, density=True)  # normalized to probability
```

bins parameter

Why: Controls how many bars. Too few = lose detail. Too many = too noisy. Try 10–50.

```
plt.hist(ages, bins=10)   # few groups
plt.hist(ages, bins=50)   # many groups
plt.hist(ages, bins=[20,30,40,50,60])  # custom
```

density=True vs frequency

Why: density=True shows probability density (area=1). False shows raw counts.

```
plt.hist(data, density=False)  # count (default)
plt.hist(data, density=True)   # probability
plt.hist(data, cumulative=True) # cumulative
```

S6 · Scatter Plot & Trend Line — plt.scatter()

plt.scatter(x, y, ...)

Why: Show relationship between two numeric variables. Use to find correlation — spend vs sales.

```
plt.scatter(x, y)                                # basic
plt.scatter(x, y, c='coral', s=100, alpha=0.7) # styled
plt.scatter(x, y, c=colors, s=sizes, cmap='viridis')  # colored by 3rd var
```

s — Marker Size

Why: s sets dot size. Use larger dots for fewer points, smaller for dense datasets.

```
plt.scatter(x, y, s=50)    # small dots
plt.scatter(x, y, s=200)   # large dots
plt.scatter(x, y, s=sizes) # size by value
```

alpha — Transparency

Why: When dots overlap, use alpha (0=invisible, 1=solid) to see density.

```
plt.scatter(x, y, alpha=0.3)  # very transparent
plt.scatter(x, y, alpha=0.7)  # slightly transpa
rent
plt.scatter(x, y, alpha=1.0)  # solid (default)
```

Trend Line — np.polyfit()

Why: Add a best-fit line over scatter plot to show the overall direction of the relationship.

```
import numpy as np
m, b = np.polyfit(x, y, 1)      # 1 = linear fit
plt.scatter(x, y, alpha=0.6)
plt.plot(x, m*np.array(x)+b, color='red', label='Trend')
```

S7 · Subplots & Multi-Panel Figures

plt.subplots(rows, cols)

Why: Create grid of multiple plots in one figure. Use to compare multiple charts side by side.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))  # 1 row 2 cols
axes[0].plot(x, y1)   axes[0].set_title('Chart 1')
axes[1].bar(x, y2)    axes[1].set_title('Chart 2')
plt.tight_layout()   # prevents overlap
```

2D Grid of Subplots

Why: For a 2×2 or 2×3 grid. axes becomes a 2D array — access with axes[row][col].

```
fig, axes = plt.subplots(2, 2, figsize=(10,8))
axes[0][0].plot(x, y1)
axes[0][1].bar(x, y2)
axes[1][0].scatter(x, y3)
```

Shared Axes

Why: sharex/sharey ensures all subplots have same scale — great for fair comparison.

```
fig, ax = plt.subplots(1,2,
    figsize=(10,5), sharey=True)
ax[0].plot(x, y1)
ax[1].plot(x, y2)  # same y scale
```

plt.tight_layout()

Why: Auto-adjusts spacing between subplots to prevent title/label overlap.

```
plt.tight_layout()           # default
plt.tight_layout(pad=2.0)    # more padding
```

plt.subplot() — Old Way

Why: Older method. plt.subplot(rows, cols, index). Index starts at 1.

```
plt.subplot(1, 2, 1)  # row1 col1
plt.plot(x, y1)
plt.subplot(1, 2, 2)  # row1 col2
plt.plot(x, y2)
```

S8 · Customization & Style Management

plt.style.use('style_name')

Why: Apply a complete visual theme to all plots. Use 'seaborn', 'ggplot', 'fivethirtyeight' for pro look.

```
plt.style.use('seaborn-v0_8')         # seaborn style
plt.style.use('ggplot')               # R ggplot style
plt.style.use('fivethirtyeight')      # news style
print(plt.style.available)            # see all options
```

Figure Size & DPI

Why: figsize sets dimensions in inches. DPI controls resolution (72=screen, 300=print).

```
fig = plt.figure(figsize=(12, 6), dpi=100)
# width=12in, height=6in, 100dpi
plt.figure(figsize=(8,4))  # shortcut
```

Line Styles Quick Ref

Why: Combine color + linestyle + marker in one string shortcode.

```
plt.plot(x, y, 'r--')    # red dashed
plt.plot(x, y, 'bo-')    # blue solid+circle
plt.plot(x, y, 'g^:')    # green dotted+triangle
```

plt.grid()

Why: Add background grid lines for easier reading of values.

```
plt.grid(True)                    # default grid
plt.grid(axis='y', alpha=0.5)     # y-axis only
plt.grid(linestyle='--', color='gray')
```

Color Palettes

Why: Set consistent colors for professional charts.

```
colors = ['#2196F3','#F44336','#4CAF50']
plt.bar(x, y, color=colors)
# or use colormap:
cmap = plt.cm.get_cmap('tab10')
```

S9 · Advanced Charts — Stacked, Area, Error Bars, Time Series

Stacked Bar Chart

Why: Show total AND composition of each category. Good for sales breakdown by product/region.

```
plt.bar(x, bottom_vals, label='Product A')
plt.bar(x, top_vals, bottom=bottom_vals, label='Product B')
# 'bottom' stacks second bar ON TOP of first
plt.legend()
```

Area Chart — plt.fill_between()

Why: Filled line chart. Shows volume over time — website traffic, revenue accumulation.

```
plt.plot(x, y, color='blue')
plt.fill_between(x, y, alpha=0.3, color='blue')
# fill between line and x-axis
```

Error Bars — plt.errorbar()

Why: Show uncertainty or variability around data points — confidence intervals, std dev.

```
plt.errorbar(x, y,
    yerr=std_values,
    fmt='o', capsize=5,
    ecolor='red')
```

Time Series Charts

Why: Plot data over dates/time. Combine with pandas for real-world time-indexed data.

```
import pandas as pd
df['date'] = pd.to_datetime(df['date'])
plt.plot(df['date'], df['sales'])
plt.gcf().autofmt_xdate()  # auto rotate date labels
```

S10 · Annotations, Legends & Highlighting

plt.annotate(text, xy, xytext, ...)

Why: Add arrows and text to highlight specific data points — peak sales, anomalies.

```
plt.annotate('Peak!',
    xy=(x_peak, y_peak),          # point to annotate
    xytext=(x_peak+1, y_peak+5), # where text goes
    arrowprops=dict(arrowstyle='->', color='red'),
    fontsize=12, color='red')
```

plt.text(x, y, text)

Why: Add plain text anywhere on the chart — no arrow. Good for chart subtitles or notes.

```
plt.text(0.5, 0.9, 'Max Revenue',
    transform=plt.gca().transAxes,
    fontsize=12, ha='center')
# transAxes: 0-1 scale, not data coords
```

plt.legend()

Why: Show legend box for labeled series. Essential when multiple lines/bars exist.

```
plt.plot(x, y1, label='2023')
plt.plot(x, y2, label='2024')
plt.legend(loc='upper left')
# loc: upper/lower left/right/center
```

Horizontal / Vertical Lines

Why: Mark thresholds, averages, or targets with reference lines.

```
plt.axhline(y=mean_val, color='red',
    linestyle='--', label='Mean')
plt.axvline(x=target_x, color='green',
    linestyle=':')
```

plt.xlim() / plt.ylim()

Why: Set exact range of x and y axes. Use to zoom into a region.

```
plt.xlim(0, 100)       # x from 0 to 100
plt.ylim(-10, 500)     # y range
ax.set_xlim([jan, dec]) # on axes object
```

S11 · Exporting & Saving Plots — plt.savefig()

plt.savefig('filename.ext', ...)

Why: Save plot to file. Call BEFORE plt.show(). Supports PNG, SVG, PDF, JPEG formats.

```
plt.savefig('chart.png')                          # basic PNG
plt.savefig('chart.pdf', dpi=300)                 # high-res PDF
plt.savefig('chart.svg', bbox_inches='tight')     # vector SVG
plt.savefig('chart.jpeg', dpi=150, quality=90)    # JPEG
```

DPI — Resolution Control

Why: DPI = Dots Per Inch. 72=web, 150=presentations, 300=print quality.

```
plt.savefig('web.png', dpi=72)    # web
plt.savefig('ppt.png', dpi=150)   # PPT
plt.savefig('print.pdf', dpi=300) # print
```

bbox_inches='tight'

Why: Removes extra whitespace around the figure so it looks clean when embedded.

```
plt.savefig('clean.png',
    bbox_inches='tight',
    transparent=True)  # transparent bg
```

Format Comparison

Why: PNG for web/PPT, SVG for scaling, PDF for documents, JPEG for photos.

```
# PNG  - raster, good for screens
# SVG  - vector, scales perfectly
# PDF  - vector, best for reports
# JPEG - raster, smaller file size
```

Save to Memory (BytesIO)

Why: Save chart to memory buffer — useful for embedding in Flask/Django web apps.

```
from io import BytesIO
buf = BytesIO()
plt.savefig(buf, format='png')
buf.seek(0)  # ready to use
```

■ Quick Reference — All Functions at a Glance

Function	What it Does	When to Use
`plt.plot(x,y)`	Line chart	Trends over time
`plt.bar(x,y)`	Vertical bar chart	Compare categories
`plt.barh(x,y)`	Horizontal bar chart	Long category names
`plt.pie(sizes)`	Pie chart	Part-to-whole %
`plt.hist(data,bins)`	Histogram / distribution	Show data spread
`plt.scatter(x,y)`	Scatter plot	Find correlation
`plt.fill_between(x,y)`	Area / filled chart	Volume over time
`plt.errorbar(x,y,yerr)`	Error bar chart	Show uncertainty
`plt.subplots(r,c)`	Multiple charts grid	Compare side by side
`plt.annotate(txt,xy)`	Arrow + text annotation	Highlight data point
`plt.text(x,y,txt)`	Plain text on chart	Add notes/labels
`plt.legend()`	Show legend box	Multiple series
`plt.title()`	Chart title	Always — clarity
`plt.xlabel/ylabel()`	Axis labels	Always — clarity
`plt.xticks/yticks()`	Customize tick labels	Rotation/formatting

`plt.grid()`	Background grid lines	Easier value reading
`plt.axhline/axvline()`	Horizontal/vertical line	Mark threshold/avg
`plt.xlim/ylim()`	Set axis range	Zoom into region
`plt.style.use()`	Apply visual theme	Professional look
`plt.tight_layout()`	Fix subplot spacing	After subplots
`plt.savefig()`	Export to PNG/PDF/SVG	Save final chart
`plt.show()`	Display the plot	End of every script
`np.polyfit(x,y,1)`	Linear trend line	Add best-fit line
`pd.to_datetime()`	Convert dates	Time series charts